

ASSIGNMENT REPORT-3

Submitted by :

KARTHIKEYAN M [26120004]

MIRTHULA R [261200121]

Under the guidance of :

HEMA ARUNACHALAM MURTHY

For The Subject

Pattern Recognition and Machine Learning

(26AI5707)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SHIV NADAR UNIVERSITY

KALAVAKAM

AUGUST 2026

INTRODUCTION

This assignment is mainly about Polynomial Regression. We were given a data with non-linear relationship. The assignment requires us to utilize the help of polynomial basis functions and create a design matrix and transform the inputs and then learn the weights in the transformed space. We will look into that visually later. We have also performed some extra experiments like trying out what happens when you don't normalize your data and also tried a different basis function like Gaussian Basis, that has been asked in the last short exam.

DATA AND SPLIT

Table 1: Dataset and split used for model selection.

Set	Samples	Percentage
Training	6000	60%
Validation	2000	20%
Test	2000	20%

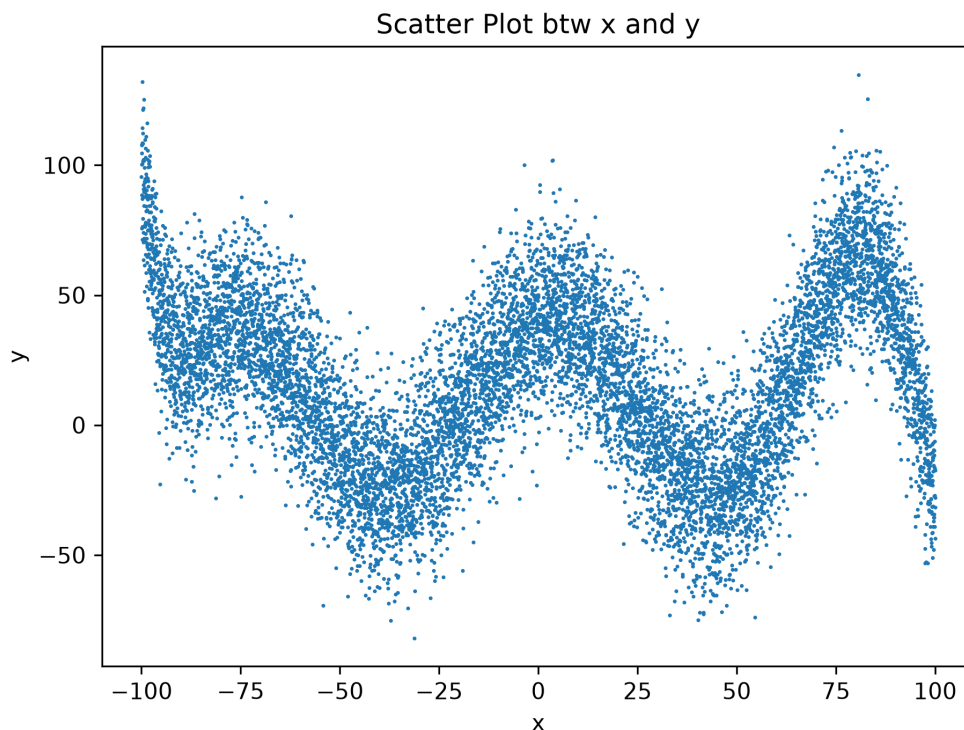


Figure 1: Scatter plot of the 10,000 input–target pairs.

The given data are clearly non-linear in nature. Thus, a normal linear regression would not be sufficient. Also, when splitting the shuffling part is crucial because without it we would

get cut off within a range and will never get the full representation of the data points. Thus, the samples were shuffled with seed 42 before splitting.

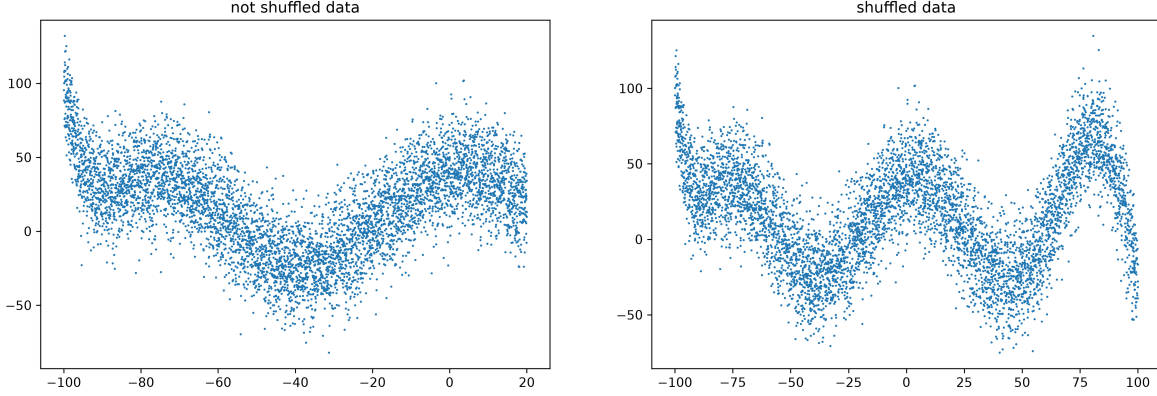


Figure 2: Sample order before and after shuffling.

POLYNOMIAL REGRESSION

Basis and least-squares solution

For degree d , the design matrix looks something like this and the each feature is standardized.

$$\phi(z) = [1 \quad z \quad z^2 \quad \dots \quad z^d]^\top, \quad z = \frac{x - \mu_x}{\sigma_x}. \quad (1)$$

The target variable is also scaled to match the input scale $t = (y - \mu_y)/\sigma_y$.

$$\Phi_{nj} = \phi_j(z_n), \quad \hat{t}(z, \mathbf{w}) = \mathbf{w}^\top \phi(z). \quad (2)$$

The weights are obtained by solving for least squares:

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t}. \quad (3)$$

We start with a single input vector $\mathbf{x}$ and output y . We then standardize the data, We transform our inputs using a polynomial basis and then we solve for the least squares solution. We will talk about normalization in more detail in the additional experiments section.

Degree selection

We experimented different degrees of polynomial (i.e.) from 1 to 15 in order to choose the best degree of polynomial, which is decided via calculating MSE (Mean Squared Error) for each one of those degrees.

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2. \quad (4)$$

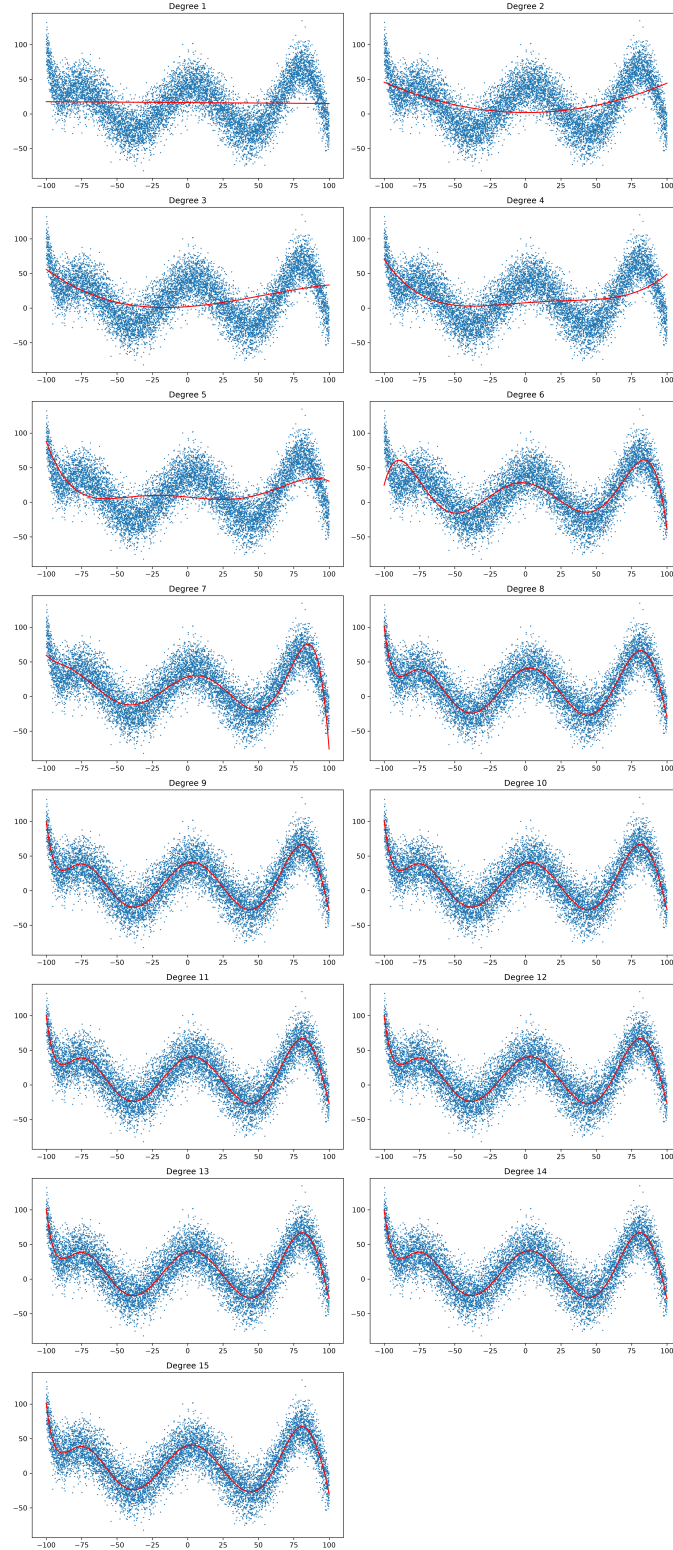


Figure 3: Polynomial fits for degrees 1 to 15.

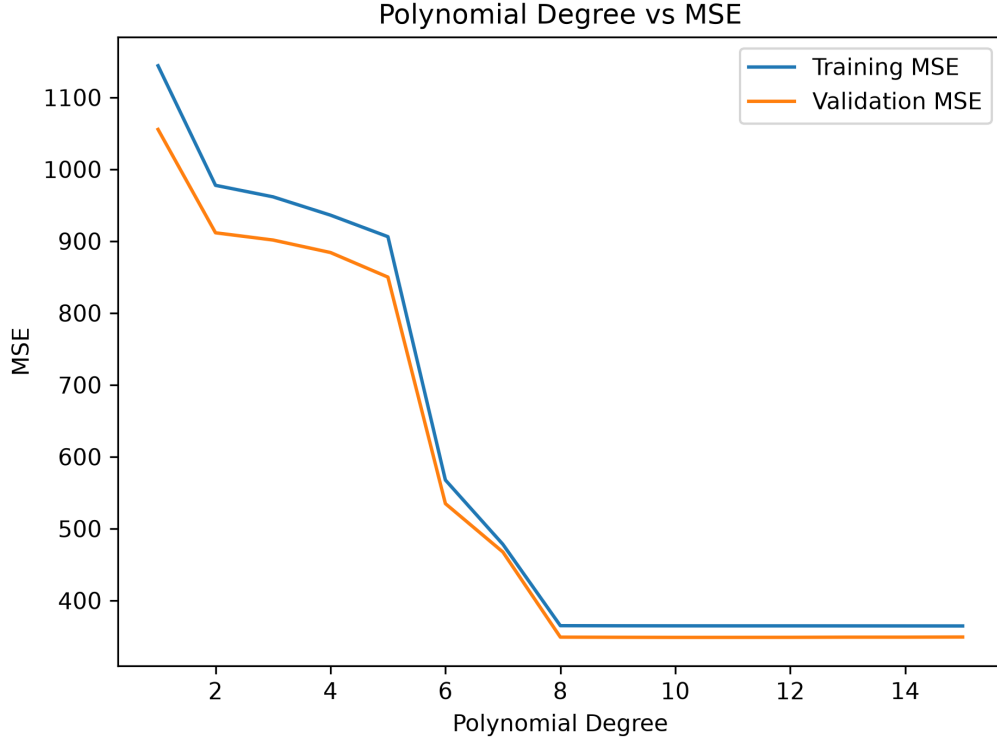


Figure 4: Training and validation MSE for each polynomial degree.

Table 2: Selected values from the degree comparison.

Degree	Training MSE	Validation MSE
1	1144.10	1055.51
5	906.23	849.98
8	364.92	348.96
10	364.69	348.63
15	364.53	349.14

We train the model on the train set and then we try out different degrees of polynomial basis on the validation set. Out of what we have tested above, Polynomial of degree 10 gave the lowest validation MSE. Thus, we choose degree 10 as the best polynomial degree.

Final evaluation

The polynomial of degree 10 is modeled on the combined set of training and validation examples (8000 samples). That yields us Test MSE of 340.0071.

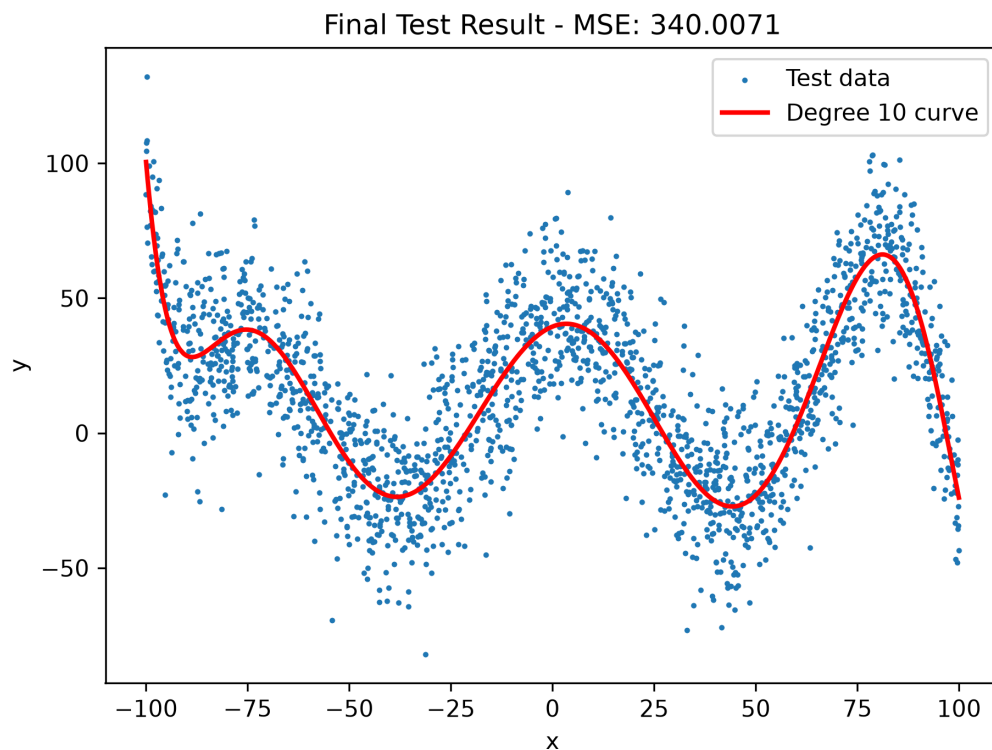


Figure 5: Final degree-10 polynomial fit on the test set.

ADDITIONAL EXPERIMENTS

Polynomial basis without normalization

The degree and the 80–20 final split were kept fixed. The unnormalized degree-10 model produced a test MSE of 340.0071.

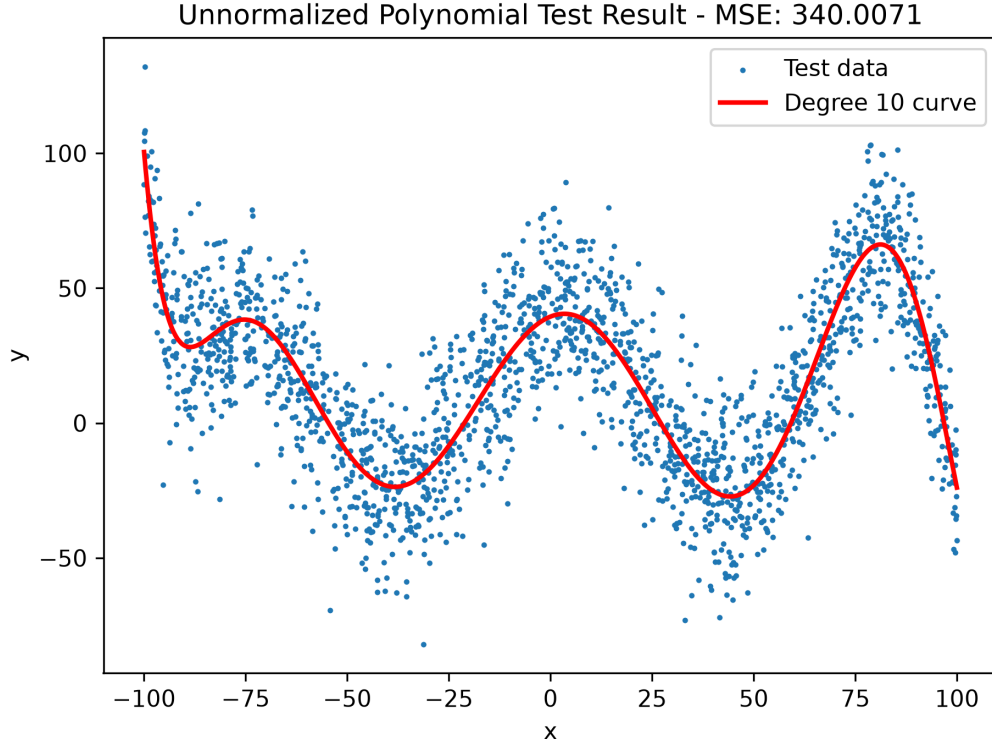


Figure 6: Degree-10 polynomial fit without normalizing the input or target.

Table 3: Examples of weights from the unnormalized polynomial model.

Weight	Value
w_0	3.947×10^1
w_5	1.623×10^{-7}
w_{10}	2.457×10^{-18}

Both normalized and unnormalized version of polynomial regression yields us the same results. But, there is one particular thing that needs attention (i.e.) the weights. In the unnormalized version the weights had to get smaller in order to compensate for the large magnitude of input features as you can see w_{10} is very small. But, the whole point of doing normalization is to improve the numerical stability while computation of least squares solutions. Because things like $\Phi^T \Phi$ can easily have large values and that might cause problems like cutting of some floating point precision and etc... Usually it's a good thing to scale the values before you move on to modeling.

Gaussian basis

For center c_j and fixed width σ , the Gaussian basis function is

$$\phi_j(x) = \exp \left\{ -\frac{(x - c_j)^2}{2\sigma^2} \right\}. \quad (5)$$

The design matrix is constructed with gaussian basis functions and equally spaced centers are chosen and we experiment with multiple center counts such as 5, 10, 15, 20, 25, 30, and 35.

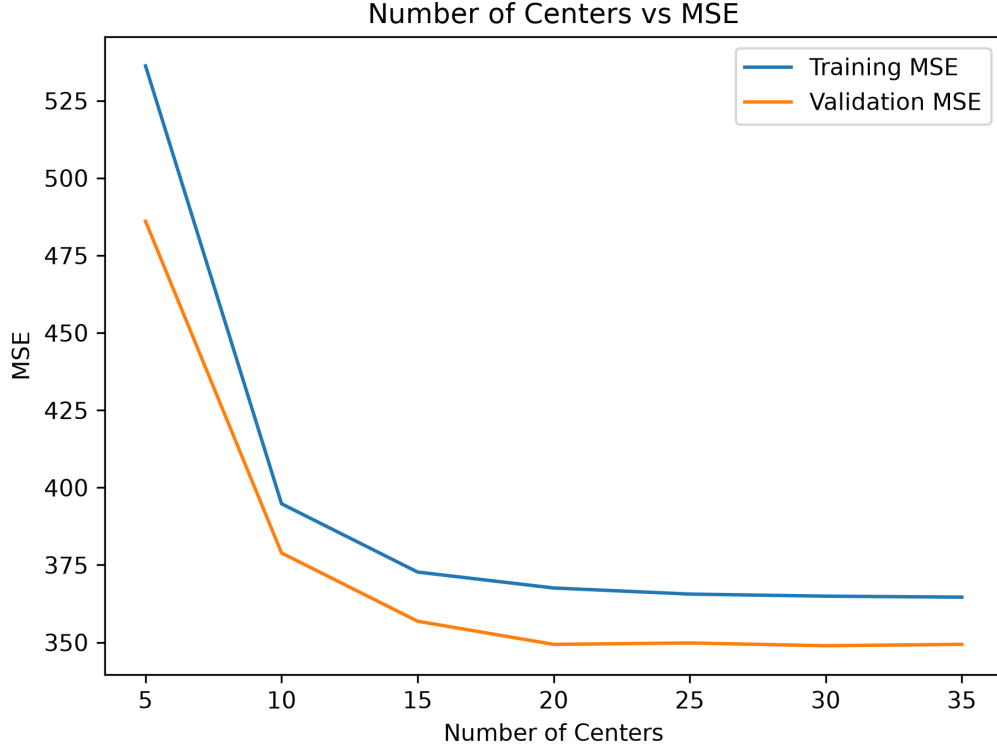


Figure 7: Training and validation MSE for the candidate Gaussian centre counts.

The validation MSE becomes lowest when 348.9237 at 30 centers. After combining the train and validation set we re-train the Gaussian model and obtain a test MSE of 342.1092.

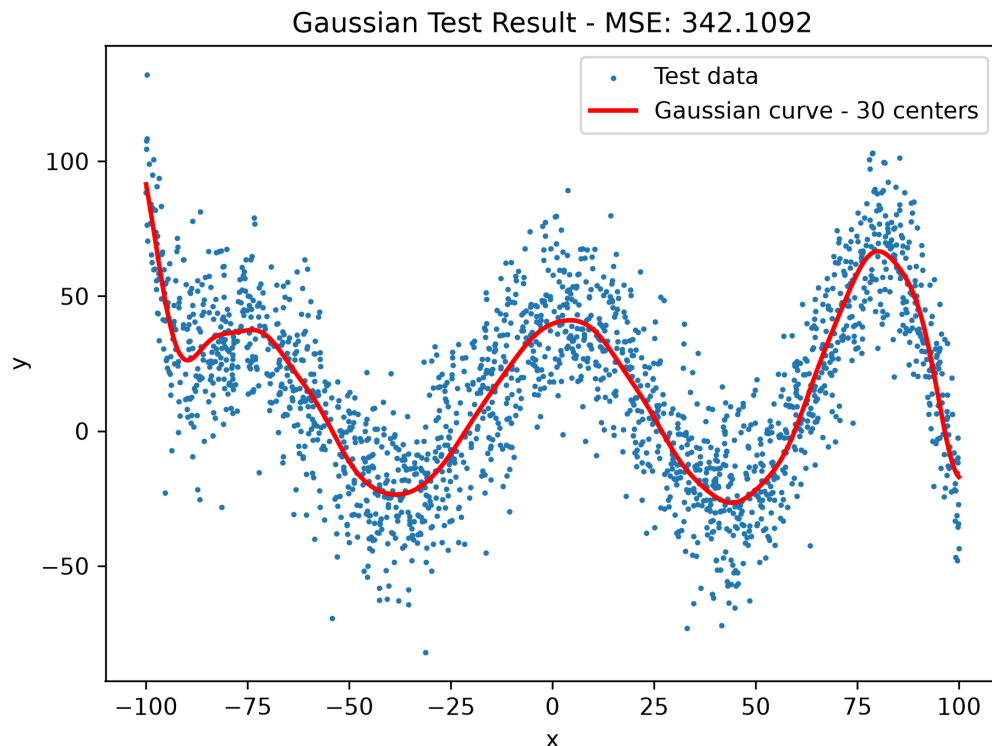


Figure 8: Final 30-center Gaussian-basis fit on the test set.

In gaussian basis it's all about centers and distances. If a point is away from the center it returns a low value and if a point is near the center it returns a high value. While the MSE came out to be pretty close to the polynomial basis, Visually the gaussian basis is very close to what polynomial basis have achieved. But from what we have tested, we could conclude among the 2 basis, Polynomial basis is performing better in this data.

INFERENCE

Table 4: Final comparison of the three fitted models.

Model	Selected size	Test MSE
Normalized polynomial	Degree 10	340.0071
Unnormalized polynomial	Degree 10	340.0071
Gaussian basis	30 centers	342.1092

The polynomial with the degree 10 learned the relationship better between the inputs and the outputs. Normalization doesn't particularly improve the the model. But, it changed the scale of the weights, which is expected as weights are just compensations that gets multiplied with inputs to reach the output. Gaussian basis with 30 centered performed considerably well. Polynomial basis performed slightly better while tested.

Overall this experiment is a combination of learning new things and clearing out misconceptions. We learned how weights compensate for the inputs magnitude. Also, our misconceptions in the area of basis functions has been cleared. Like before we thought input feature matrix itself has multiple columns and they each get transformed via the basis function and get placed in their respective places. After this experiment, we came to know that a single feature vector is transformed into multiple features via basis function and that is a valuable learning experience.

References

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Chapter 3.